

Research Paper

Title	Analysing Query Embedding Spaces in RAG-Based Text-to-SQL Systems Using PCA and SVM
Core gap	The geometric structure of this embedding space is uncharacterised
Research Objectives	Find the most efficient mathematical representation of query data using PCA. Build an intelligent router (SVM) to sort queries by difficulty.

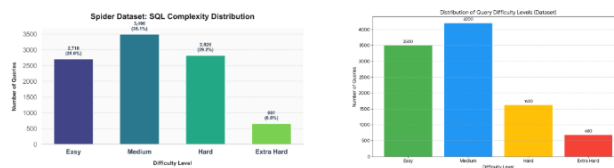
My research investigates the geometric properties of query embedding spaces to optimize Text-to-SQL systems. I began working on an academic benchmark, and have since migrated the entire pipeline to Sumvec's **768-dimensional production architecture**.

PCA - technique of DR. looking at the most meaningful layers of information and compressing the data to those no. of dim

SVM - method that allows to find the most optimal hyperplane that separates the data based on inherent properties of difficulty level

Datasets

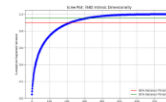
Benchmark dataset: Spider	Production Database: Pinecone
10,000 NLO-SQL pairs	11,435 Question-SQL pairs
384-dimensional vectors	768-dimensional vectors
Difficulty levels: Easy / Medium / Hard/ Extra Hard	
Heuristics: JOINS, GROUP BY, nests	



The data was initially present in a raw format and was converted to 384 dimensional vector space using the BAAI model. Next we look at the mathematical efficiency discovered during this process.

Results: Dimensionality

- Standard PCA method chosen: fastest and most reliable for text embeddings
- Analysed 768 production dimensions
- 90% variance captured in 220 components



Dimensionality Reduction Variant Comparison

	Standard PCA	Incremental PCA	Kernel PCA	Sparsity PCA
Time (s)	4.2 (1s)	1.8 (s)	30 (mins)	30 (mins)
Loss	6.16e-15	7.10e-14	1.00e-10	6.08e-10
Variance Explained (%)	91.41	91.4	94	94

These 4 were chosen because they represent the 4 fundamental ways to **handle linear vs. non-linear data structures**.

The 10% loss is mostly "noise"—tiny variations in phrasing or punctuation that don't change the actual meaning of the question.

Y-axis: gives the amount of info captured as a decimal

Initial steep increase: first few dimensions capture the larger patterns

The later flattening means we are just adding tiny details that don't help much.

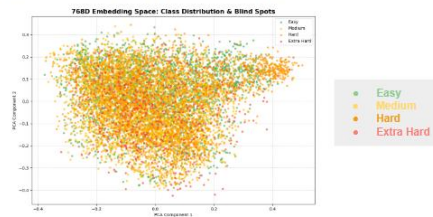
Blue line: cumulative signal as we add more PCA components

Results: Subspace and Blind Spots

Retained 90% of information while simplifying from 768 dimensions.

- Easy queries are tightly clustered.
- Extra Hard queries are geometrically dispersed.

RESOLUTION: augmenting the index with diverse nested-query examples



Axes represent the two strongest mathematical "directions" of variation in the text. Linear transformation is used to see the 2D projection of the 768 D space. (ANALOGY: light is shone on a 768 D object and we are looking at its 2D shadow.)

Based on these shapes, we needed a tool to draw boundaries between these groups.

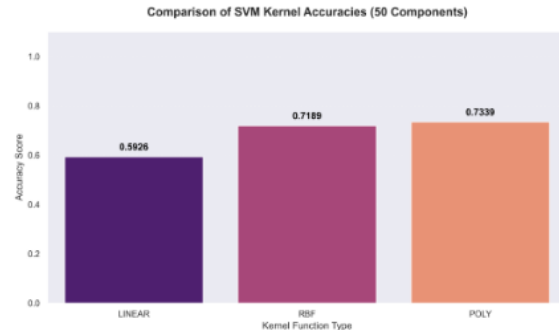
Results: Kernel and Embedding Geometry

Radial Basis Function (RBF) Kernel chosen

- captures curved, complex boundaries
- Linguistic overlap between classes (especially between 'Easy' and 'Medium') is handled effectively via non-linear hyperplane.
- performs consistently across all component counts and much more stable in production

POLY kernel - more complex to tune and slower

Kernel Type	Accuracy	Execution Time (seconds)
Linear	0.5926 (59.26%)	3.99s
Poly	0.7189 (71.89%)	5.63s
RBF	0.7339 (73.39%)	3.83s



From the previous slide we saw that the data groups **cannot be separated by straight lines**. Hence the **LINEAR** kernel cannot be used.

RBF uses a "local" approach. It prioritises nearby points, which is perfect for capturing the **tight "pockets"** of Easy queries.

works best for sorting these questions in the real world.

Though POLY performs well at specific points, ..

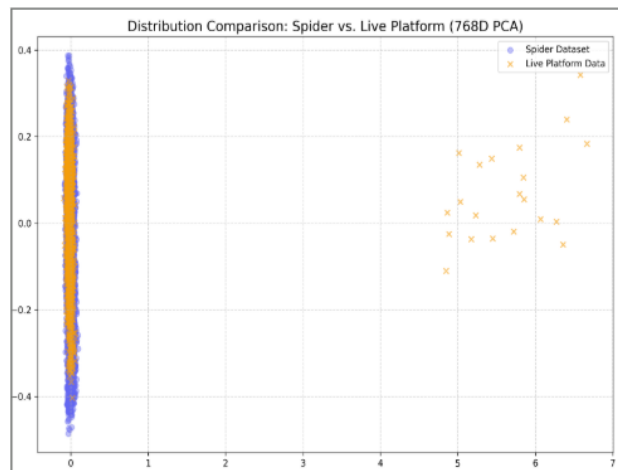
POLY is inconsistent with live traffic

Finally, let's compare our academic baseline with the live platform.

Results: Classification Performance

- **10-point** jump in F1-Score achieved on production vectors
- Viable routing layer for live traffic
- Higher dimensional production embeddings provide superior linear separability.

Metric	384D (Spider dataset)	768D (Production Sandbox)
SVM F1 Score	0.71	0.81



X-Axis represents the **Sample Count**. In this plot, the queries are ordered along the axis to show their spread across the dataset.

Y-Axis represents the **relative semantic position** given by a similarity score. It measures how "far" each query is from the mean query

Points on the Left: typical, common queries. Points on Right: (outliers) unusual queries **seen in real world but not in academic data**.

Overlap part: speak the **same language. 90% match**

This plot proves that our **research is valid**.

Let's have a look at what this means for the production architecture.

Implications for Production Architecture

- **Standard PCA** is now the justified, evidence-backed choice for Pinecone.
- **Routing layer works**: 'Easy'/'Medium' queries are routed to free models from Openrouter, handling **80% of the traffic**. While the 'Hard'/'Extra Hard' queries are directed to Claude Haiku.
- PCA results identify a **71% redundancy** in the embedding space; eliminating this noise through dimensionality reduction guarantees **faster similarity search retrieval** and **lower cloud infrastructure costs**.

Feature	Original (Without PCA)	Reduced (With PCA)	Savings
Dimensions	768	220	~71% Reduction
Data Size (Est.)	37.2 MB	10.9 MB	26.3 MB saved